

Compilador “Mini-LANG”

Integrantes: Gerson F. Dutra [2020003148] | Lilian Suyane [2019003109]

Repositório: github.com/GersonFeDutra/Compilador-Mini-Lang

Mini-LANG: Linguagem imperativa, procedural e estaticamente tipada.

Linguagem do compilador: Python

Linguagem alvo: Python

Instruções de execução: execute o programa com o comando:

```
python -m src.compiler -help
```

para obter opções de compilação.

Cada módulo também pode ser executado independente. Exemplo:

```
python src.modules.lexer -help
```

A Linguagem Mini-Lang

Suporta:

- **coerção** de tipos:
 - $\text{int} \rightarrow \text{real}$ (regra do tipo maior)
 - $\text{real} \rightarrow \text{int}$ (truncamento ao declarar variável)
 - $\text{bool} \rightarrow \text{int}$ (contagem)
 - $\text{int} \rightarrow \text{bool}$ (regra da “veracidade”: $\text{False} \Leftrightarrow 0$)
- tipos **incompatíveis**:
 - $\text{qualquer} \leftrightarrow^1 \text{string}$
 - $\text{bool} \leftrightarrow \text{real}$
- detecção de sombreamento (**shadowing**) de variáveis e parâmetros de função.

Exemplos de código são encontrados em ./samples

A linguagem Mini-Lang será descrita formalmente abaixo.

Estrutura Gramatical EBNF

A gramática apresentada é descrita utilizando a notação EBNF (Extended Backus-Naur Form). Ela é uma extensão do BNF clássica que permite descrever regras gramaticais de forma mais concisa e legível.

1. Traduzindo Repetições: $\{ A \}$ A notação $\{ X \}$, significa uma lista de X 's (podendo ser vazia). Em EBNF, isso vira uma recursão $S \rightarrow A \{ B \} C$, já em BNF isso seria escrito como:

$$\begin{aligned} S &\rightarrow AXC \\ X &\rightarrow BX \mid \epsilon \end{aligned}$$

¹ um para o outro, e vice-versa

2. Traduzindo Opcionalidade: [A]: A notação [X], indica que X pode aparecer ou não. Em EBNF, isso seria representado como:

$$S \rightarrow \text{if}(E)S [\text{else } S]$$

Já em BNF escrevemos:

$$\begin{aligned} S &\rightarrow \text{if}(E)SX \\ X &\rightarrow \text{else } S \mid \epsilon \end{aligned}$$

3. Traduzindo Agrupamento: (A | B): Os parênteses servem para controlar a prioridade das escolhas. Em EBNF

$$\text{Factor} \rightarrow (+|-) \text{Term},$$

em BFN:

$$\begin{aligned} \text{Factor} &\rightarrow \text{Op Term} \\ \text{OP} &\rightarrow + \mid - \end{aligned}$$

Nota: Trechos marcados em verde são adições em relação à gramática Mini-Lang original e em azul estão são mudanças onde as diferenças estão listadas nas notas de rodapé (o número sobrescrito nestes caso é a referência para a nota correspondente).

```
< program > = { symTable=null; } { < statement > }
< statement > = < variable-decl > ","
    | < assignment > ";"
    | < print - statement > ";"
    | < if - statement >
    | < while - statement >
    | < return - statement > ";"
    | < function - decl >
    | < block >
    | “,”
< block > = "{" { < statement > } "}"

<function-decl> = "def " <identifier> "(" [ <formal-params> ] ")" " ":" <type> <block>
<formal-params> = <formal-param> { "," <formal-param> }
<formal-param> = <identifier> ":" <type>

<return-statement> = "return" <expression>
<print-statement> = "print" < string - literal > | <expression>

< variable - decl > = " var " < identifier > ":" < type > "=" < expression >
< assignment > = " set " < identifier > "=" < expression >

<while-statement> = "while" "(" <expression> ")" <block>
<if - statement > = " if " "(" < expression > ")" " < block > [ " else " < block > ]

<expression> = <simple-expression> { <relational-op> <simple-expression> }
<simple-expression> = <term> { <additive-op> <term> }
<term> = <factor> { <multiplicative-op> <factor> }
```

```

< factor > = < literal >
            | < identifier >
            | < function - call >
            | <sub - expression >
            | < unary >

<unary> = ( "+" | "-" | "not" ) { < expression > }
<sub - expression > = "(" < expression > ")"
<function - call> = < identifier > "(" [ < actual - params > ] ")"
<actual - params> = < expression > { "," < expression > }

<relational - op > = "<" | ">" | "==" | "!=" | "<=" | ">="
<additive - op > = "+" | "-" | " or "
<multiplicative - op > = "*" | "/" | " and "

< literal > = < integer - literal >
            | < real - literal >
            | " true "
            | " false "
<integer-literal> = [0-9]+
<real-literal> = <digit>+ "." <digit>+
<string-literal> = "" { < letter > | < digit > | < symbol > | " " } ""

< type > = " int " | " real " | " bool " | " void "
<identifier> = ( "_" | < letter > ) { "_" | < letter > | < digit > }
<digit> = [0-9]
<letter> = [a-zA-Z]
<symbol> = <any-char-except-quote> = (Qualquer caractere imprimível , exceto aspas
Σ\{ " })

```

Esquema de Tradução com EBNF

Nesse esquema de traduções, **ações semânticas** são marcadas em negrito (entre chaves - diferente dos lexemas de fechamento que representam repetições). Exemplos de ações:

- **warn** <message>: indica um log de aviso
- **error** <type> <message>: indica uma falha na compilação (de tipo sintática ou semântica) - encerra o processo imediatamente.
- **match** <lexema>: verifica uma correspondência de lexemas. Resulta em erro de tipo sintático no caso de falha.
- Sempre que um “literal” (entre aspas) aparecer no esquema, existe uma ação **match** implícita com erro no formato
“Err. sint. <arquivo>:<linha>:<coluna>: esperado <lexema> no <contexto>” O <contexto> é adicionado conforme a implementação.
- **compatible**: condição que verifica se os tipos são compatíveis

Uma ação semântica dentro de um fechamento (entre chaves e sem negrito), significa que a ação será aplicada para cada token processado - De forma semelhante, uma ação semântica **entre chaves da notação opcional** significa que ela só será aplicada se o token for correspondido.

A expressão semântica [**<operação> for i in <vetor>**] é um for-inline que, semelhante ao python, permite modificar cada item do <vetor> aplicando a <operação> em cada item i, onde i é a variável temporária que representa um único item.

Variáveis e notação adicional:

- **l: lookahead** - token corrente
- **\$:** token de final do arquivo
- **st:** *symbol table* - tabela de símbolos
- **ln:cl:** coordenadas - tupla formado por linha (**ln**) e coluna (**cl**), ambas variáveis
- **outras:** se não foram mencionadas aqui são variáveis temporárias no escopo daquela regra.

Alguns tokens são **abreviados** na ação semântica para expressar a gramática de forma mais compacta. Eg.: <formal-param> → <f-p>

A operação de acesso “.” após o token, é equivalente no modelo orientado a objetos, onde o termo após o “.” é um atributo deste token. Eg.: <f-p>.node

- Cada token carrega um parâmetro **node ou nodes** (vetor).
- Se não for explicitamente mencionado, o node de uma variável filha (eg.: <block>) é repassado para o node de uma variável pai (eg.: <statement>) com o tipo correspondente Block → Statement.

```

< program > → { st=null; } { < statement > }
               { if (l != $) error syntax "unknown token" }
< statement > → < variable-decl > ";"
               | < assignment > ";"
               | < print-statement > ";"
               | < if - statement >
               | < while - statement >
               | < return - statement > ";"
               | < function - decl >
               | < block >
               | “;” { warn “empty statement” }

< block > → "{" { saved= st; st = SymTable(previous=st); stmts=[] }
              { <statementi> { stmts.append(<si } } "}"
              { st = saved; <b>.node=Block(stmts) }
<function-decl> → "def" { name=str(l) } <identifier>
                  "(" { saved=st; st=SymTable(previous=st) }
                  [ <formal-params1> { params=<f-ps1 } ] ")" ":" { type=str(l) }
                  <type>{s= Symbol(name, [Symbol(p.name, p.type) for p in
params]);
                  st.insert(s) } <block1> { st = saved; st.insert(s);
                  <f-d>.node = FunctionDecl(name, params, type, body=<b1
                  }

```

```

<formal-params> → { params=[] } <formal-param1> { params.append(<f-p1>.node) }
                { "," <formal-param1> { params.append(<f-p1>.node) } } }
                { <f-ps>.nodes=params }

```

```

<formal-param> → { name=str(l) } <identifier> ":" { type=str(l) } <type>
                { if !( st.insert(name, Symbol(name, type, ln:cl)) )
                  error syntax "parameter exits";
                  if st.is_shadowing(name)
                    warn "parameter shadowing symbol";
                  <f-p>.node=FormalParam(name, type) }

```

```

<return-statement> → "return" <expression>

```

```

<print-statement> → "print" <string-literal> | <expression>

```

```

<variable-decl> = "var" <identifier> ":" <type> "=" <expression> {
                  if ! compatible(<t>, <e>.type)
                    error semantic "incompatible types";
                  if <t> == int and <e>.type == real
                    warn "lose precision - value truncated";
                  if !( st.insert(name, Symbol(name, type, ln:cl)) )
                    error syntax "variable exits";
                  if st.is_shadowing(name)
                    warn "parameter shadowing symbol";
                  <f-p>.node=VarDecl(name, type) }

```

```

<assignment> = "set" <identifier> "=" <expression> { if ! st.has(<i>)
              error syntax "undeclared variable";
              if ! compatible(<i>, <e>.type)
                error semantic "incompatible types";
              if ( st.find(<i>).type == int <e>.type == real )
                warn "lose precision - value truncated"
              <a>.node=Assignment(<i>, value=<e>, st.find(<i>).type) }

```

```

<while-statement> → "while" "(" <expression1> ")" <block2> {
                  <w-s>.node = WhileStmt(cond=<e12>.node)}

```

```

<if-statement> = "if" "(" <expression1> { cond=<e1 } ")" <block2> {
                  t_block=<b2>.node } [ "else" <block3> { f_block<b3>.node } ]
                  { <if-s>.node = IfStmt(cond, t_block, f_block) }

```

```

<expression> → <simple-expression1> { left=<s-e1 }
              { <relational-opi> <simple-expressionj> { right=<s-ej
                left=BinOp(<r-opi>, left, right) } } }

```

```

<simple-expression> → <term1> { left=<t1 } { <additive-opi> <termj> {
                  left=BinOp(<a-opi>, left, right=<tj } }

```

```

<term> → <factor1> { left=<f1 } { <multiplicative-opi> <factorj> {
                  left=BinOp(<m-opi>, left, right=<fj } }

```

```

<factor> → <literal>

```

```

| <identifier> { if !st.has(<i>) error syntax “undeclared variable”; }
| <function-call>
| <sub-expression>
| <unary>

<unary> → ( "+" | "-" | "not" ) { < expression > }
<sub-expression> = "(" < expression > ")"
<function-call> = <identifier> "(" { if ! st.has(<i>) error syntax “undeclared func” }
    [ <actual-paramsi> { if (i > param.size)
        error syntax “function called with wrong number of args”;
        if <a-pi>.type != st.find(<i>).param[i]
            error semantic “function called with invalid arg type” } ] ")"
<actual-params> = <expression> { " ," < expression > }

<relational-op> = "<" | ">" | "==" | "!=" | "<=" | ">="
<additive-op> = "+" | "-" | "or"
<multiplicative-op> = "*" | "/" | "and"

<literal> → <integer-literal>
    | <real-literal>
    | “true”
    | “false”
<integer-literal> → <digit>2+
<real-literal> → <digit>+ “.” <digit>*
<string-literal> = “ ” { < letter > | < digit > | < symbol > | " " } “ ”

<digit> = [0-9]
<letter> = [a-zA-Z]

```

Entrega e pontuações

Definição e reconhecimento linguagem (20% da nota)

- Objetivo: Definir a linguagem e identificar os Tokens.
- Entrega:
 - ☒ ~~Código do Analisador Léxico (Scanner).~~
 - ☒ ~~O programa deve ler um arquivo fonte e imprimir a lista de tokens (ex: <ID, número>, <Assing, ‘=>’>).~~
 - ☒ ~~Arquivo contendo a especificação formal da gramática.~~
 - ☒ ~~Não será aceito o uso de ferramentas automáticas de geração de analisadores léxicos (ex: JFlex, Lex, etc). O código do scanner deve ser~~

² [0-9] → <digit>

~~implementado manualmente ou utilizando bibliotecas que não gerem código automaticamente.~~

A estrutura da linguagem (30% da nota)

- Objetivo: Reconhecer a estrutura hierárquica do código.
- Entrega:
 - ☒ ~~Código do Analisador Sintático (Parser), funcionando a partir dos tokens gerados pelo Analisador Léxico (Scanner).~~
 - ☒ ~~Implementação das classes da Árvore Sintática Abstrata (AST).~~
 - ☒ ~~O programa deve ler o código e exibir a árvore visualmente (em formato JSON).~~
 - ☒ ~~Tratar de erros sintáticos básicos (ex: “Esperado um ‘;’ na linha 20”).~~
 - ☒ ~~Não será aceito o uso de ferramentas automáticas de geração de analisadores sintáticos (ex: Yacc, Bison, ANTLR, etc). O código do parser deve ser implementado manualmente ou utilizando bibliotecas que não gerem código automaticamente.~~

Tratamento de erros semânticos (25% da Nota)

- Objetivo: Garantir que o código fonte tenha sentido lógico.
- Entrega:
 - ☒ ~~Implementação da tabela de símbolos.~~
 - ☒ ~~O compilador deve barrar erros semânticos:~~
 - ☒ ~~Uso de variável não declarada.~~
 - ☒ ~~Declaração duplicada de variável no mesmo escopo.~~
 - ☒ ~~Tipos incompatíveis (ex: somar bool com um float).~~
 - ☒ ~~Verificar compatibilidade de tipos em atribuições e operações (ex: impedir soma de um bool e um float).~~
 - ☒ ~~Verificar o funcionamento nas chamadas de função e outras operações~~
 - ☒ ~~Verificação de tipos durante operações (binária e unária)~~
 - ☒ ~~Truncamento (conversão entre tipos)~~
 - ☒ ~~Garantir que funções sejam chamadas com o número de tipos corretos de argumentos.~~
- Extras:
 - ☒ ~~Coordenadas (linha e coluna) dos lexemas nas mensagens de log indicado.~~
 - ☒ ~~Deteção de sobreposição:~~
 - ☒ ~~*shadowing variables* na declaração de variáveis~~
 - ☒ ~~*shadowing* na criação de parâmetros de função~~

Funcionalidade prática do compilador (25% da Nota)

- Objetivo: Fazer o Código funcionar.
- Entrega:

- ☒ ~~Geração do código final. O compilador deve traduzir da Mini-Lang para Python ou C.~~
- ☒ ~~Caso seja gerado um código em python o mesmo deve ser executado sem erros.~~
- ☒ ~~O arquivo gerado deve ser executável e produzir a sua saída correta para o usuário.~~

Considerações Finais

- Projeto desenvolvido em Python
- Nas últimas semanas, os grupos irão apresentar seu projeto em sala e ao final será feito um teste ao vivo:
 1. O professor fornecerá 3 códigos "secrets" em mini-lang.
 2. Cada grupo irá compilar e executar esses códigos ao vivo.
 3. Critérios de sucesso:
 - (a) Compilou sem cair (crash)?
 - (b) Detectou erros semânticos (quando houver)?
 - (c) O resultado da execução está correto?